cub3D: every function, one connected story

From a .cub file to a textured first-person frame.

Presentation preparation • C / Linux / MiniLibX • source-based walkthrough

Load & validate → **Create graphics** → **Process input** → **Cast & draw** → **Release resources**

SPEAKER NOTE Opening: “We store a 2D grid, then find the first wall along each camera ray and project that hit as a textured vertical strip.”

Scope: all functions under source/, including static helpers; library boundaries and parser test harness. Library internals are summarized where relevant.
Snapshot 5a26b31 • 2026-11-06 • 95 application functions / 25 source files

How to use this pre-presentation

Part	What you should be able to explain
01 • Program and data	Startup, state ownership, map parsing, setup, and callbacks.
02 • One interactive frame	Key state, time steps, collision, camera vectors, DDA, textures, and minimap.
03 • Defense and rehearsal	Failure paths, cleanup, complexity, test evidence, and likely questions.
04 • Complete function reference	Every project function: signature, source line, purpose, callers, and direct calls.

Suggested delivery: 25–35 minutes for the flow walkthrough; allow another 20–30 minutes for the mathematical derivations. Use the reference pages to answer implementation questions. Printed page numbers and the contents page locate each module.

SPEAKER NOTE Rehearse the main story first. Then choose any function in the appendix and explain how its input came from the preceding stage.

Navigation & source map

Walkthrough: pages 4–26; mathematics: 27–39. Function reference begins on page 40. Each module below includes every public function and file-local helper.

Module	Page	Module	Page
common.c	40	game.c	41
hook.c	42	main.c	43
parser/parser.c	44	parser/parser_cells.c	45
parser/parser_color.c	46	parser/parser_config.c	47
parser/parser_grid.c	48	parser/parser_line_reader.c	49
parser/parser_lines.c	50	parser/parser_map_lines.c	51
parser/parser_reader.c	52	parser/parser_validate.c	53
player.c	54	raycast.c	55
raycast_camera.c	56	render/render_draw.c	57
render/render_fov.c	58	render/render_view.c	59
render/render_wall.c	60	runtime.c	61
screen.c	62	setup.c	63
texture.c	64		

Library boundary and test harness: page 65. Rehearsal checklist: page 66.

SPEAKER NOTE Follow the walkthrough in order; jump to a module reference when an evaluator asks for a specific helper.

The complete lifecycle

```
main(argc, argv)
 ├─ runtime_args()                          failure → return 1
 ├─ runtime_setup(&g, argv[1])               failure → cleanup → return 1
 │   ├─ zero-initialize t_game
 │   ├─ parser_load_map()
 │   └─ setup_game()
 └─ runtime_run()
     ├─ mlx_loop() → events + game_frame()
     ├─ runtime_cleanup() after loop ends
     └─ return g.state.exit_status
```

The stack variable `g` remains alive throughout the event loop. MiniLibX callbacks receive its address. Normal close requests end the loop; cleanup runs when control returns.

SPEAKER NOTE Trace a successful run before discussing details. Emphasize that runtime_close requests a stop; it does not free the game while callbacks are active.

The shared state and its ownership

Owner / type	Contents	Lifetime / role
t_game.map / t_map	grid, dimensions, four allocated paths, packed colors	Parser owns allocations until transfer; game owns them afterward.
t_game.player / t_player	position, unit direction, perpendicular camera plane	Mutable runtime values; spawn marker becomes floor.
t_game.graphics / t_graphics	MLX context, window, frame, four texture images	Created after parsing; image data addresses belong to MLX images.
t_game.state / t_game_state	keys[], view, dirty flag, last time, exit status	Zero starts V_3D, no keys, normal exit status.
Temporary calculation types	t_raycast, t_ray_state, t_ray_hit, t_column	Per-ray setup, stepping, hit result, and projected strip.
Drawing / parsing types	t_point, t_line, t_viewport; parser data/reader/line	Pixel geometry; temporary config and buffered input state.

SPEAKER NOTE Distinguish owning an MLX image pointer from borrowing its pixel address. Never free addr independently.

Parsing: a transactional pipeline

parser_load_map → **parser_read_file** → **rectangularize** → **validate** → **transfer to g**

- `runtime_args` requires exactly one argument ending in lowercase `.cub`. The reader independently checks that suffix.
- `parser_load_map` builds a zeroed local `t_parser_data`. It returns 0 on success, 1 after reporting an error.
- Reader, configuration, and validator helpers generally use 1 = success and 0 = failure. The line reader has a separate three-state contract.
- Failure frees local allocations. Success assigns `g->map = data.map`, clears the local map, replaces the spawn with `0`, and initializes the player.

SPEAKER NOTE Explain the return-value convention change explicitly. A leading ! in parser_load_map converts helper failure into the outer error path.

Buffered input and line ownership

```
parser_read_file → _parser_read_lines → parser_next_line
    → _parser_collect_line → _parser_read_byte / _parser_append_byte

parser_next_line:  1 = allocated line, 0 = EOF, -1 = error
parser_process_line: 1 = accepted, 0 = rejected
internal content status: 2 = accepted map line, ownership retained
```

- The reader fetches blocks of 4096 bytes and retries interrupted reads (EINTR). Each line starts at 128-byte capacity and doubles when needed.
- Newline bytes are consumed; a NUL terminator is added. Embedded NUL input is rejected. A final line without newline is supported.
- Configuration and blank lines are freed after processing. A stored map line remains owned by the grid. The trimmed inspection copy is always freed.
- The file is closed even when line processing fails. A close failure also rejects the read.

SPEAKER NOTE Point to the status 2 convention: it prevents freeing a string that was just stored in the map.

The parser is a small state machine

BEFORE MAP
config or blank

→

IN MAP
store original rows

→

AFTER MAP
blank lines only

Input	Action
Blank before map	Accept without starting the map.
First map-like line	Set map_started; append original text. NO/SO/WE/EA prefixes remain config candidates.
Nonblank once map started	Store as a row; the later character scan rejects configuration or illegal characters.
Blank after map started	Set map_finished. Any later nonblank input fails.
Short row	At EOF pad with spaces to maximum width. Spaces remain void, never floor.

Trimming is for classification. Leading spaces in the actual map are preserved. Tabs in a stored map row are rejected by character validation.

SPEAKER NOTE Use a blank line inside a maze as the example: the blank ends the map, so a later row is invalid.

Configuration: six required fields

```
NO textures/north.xpm
SO textures/south.xpm
WE textures/west.xpm
EA textures/east.xpm
F 50, 50, 50
C 110, 160, 210
```

- A bitmask records NO, SO, WE, EA, F and C. Repeated fields fail; configuration order is flexible before the map.
- Texture identifiers require space or tab, followed by a nonempty trimmed path. Parsing stores paths; image loading happens later.
- RGB requires exactly three unsigned decimal values, each 0–255, separated by commas. Extra characters or components fail.
- Packing: `(red << 16) | (green << 8) | blue`. Floor 50,50,50 becomes `0x323232`.

SPEAKER NOTE A valid configuration string does not prove the image can load. Explain the second validation boundary in texture_load.

Map validation: what is actually guaranteed

All six fields → At least 3 × 3 → Legal cells + 1 spawn → Check floor neighbors

- Allowed stored cells: `0`, `1`, space, and `N/S/E/W`. Exactly one player marker must exist.
- Each walkable cell must have four cardinal neighbors that are wall or walkable. Outside the grid reads as space, so exposed floor fails.
- Every walkable cell is inspected, including rooms unreachable from the player. Disconnected enclosed rooms are accepted.
- There is no flood fill or reachability requirement in this implementation. README connectivity wording is stale; current tests explicitly accept disconnected regions.

SPEAKER NOTE Say “enclosure of every walkable cell” precisely. Do not claim that all floor cells must connect to the player.

source/parser/parser_validate.c; parser_cells.c; tests/valid_maps/FF_*.cub
Snapshot 5a26b31 • 2026-11-06 • 95 application functions / 25 source files

Spawn and coordinate conventions

Spawn	Direction (x,y)	Camera plane (x,y)
N	(0, -1)	(0.6, 0)
S	(0, 1)	(-0.6, 0)
E	(1, 0)	(0, 0.6)
W	(-1, 0)	(0, -0.6)

```
World x increases to the right.
World y increases downward.
Grid access: grid[y][x]

pos = (cell_x + 0.5, cell_y + 0.5)
plane = (-dir_y, dir_x) * 0.6
```

In `maps/001.cub`, the spawn E is at zero-based cell (3,2), so position = (3.5,2.5). The grid becomes ordinary floor at that position. The horizontal field of view is approximately 2 × atan(0.6) = 61.9°.

SPEAKER NOTE Draw the camera direction and the perpendicular plane. The player is centered inside a tile, not located at a grid intersection.

Graphics setup and callback registration

_setup_mlx → texture_load → _setup_frame → _setup_hook

Resource / callback	Implementation
Context and window	mlx_init; mlx_new_window at 1024 × 768.
Four textures	XPM images; require an address, positive dimensions, and 32 bpp.
Frame	mlx_new_image; borrow addr/bpp/stride/endian; clear its bytes.
Window hooks	Key press/release, focus out, expose, and window close.
Loop hook	game_frame registered with mlx_loop_hook; all callbacks receive g.

A failed stage short-circuits setup. The caller uses the same cleanup routine for partially created resources. Hook registration stores function pointers; it does not run the callbacks immediately.

SPEAKER NOTE Make the transition from sequential startup to event-driven execution explicit.

source/setup.c; texture.c; library/mlx/mlx_hook.c; mlx_loop_hook.c
Snapshot 5a26b31 • 2026-11-06 • 95 application functions / 25 source files

Events change state; frames use that state

Event	Function and effect
W/S or up/down; A/D; left/right	common_key_id maps to logical motion; hook_press sets keys[id], hook_release clears it. Changes request redraw.
M press	Toggle only if GK_VIEW was previously clear. Latch until release; screen_toggle_view changes V_3D/V_MINIMAP and marks dirty.
Escape or window close	runtime_close calls mlx_loop_end; cleanup follows after mlx_loop returns.
Focus lost	hook_focus_out clears every logical key to prevent stuck movement.
Expose / window refresh	hook_expose requests redraw.

Callbacks are dispatched through the MiniLibX/X11 event loop. Motion is continuous because the loop callback reads held-key state, rather than moving only on a key press.

SPEAKER NOTE Mention a current edge case: W and up share one boolean. Holding both and releasing either clears forward until a later press updates it.

One frame: timing and dirty rendering

```
game_frame(g)
  if no motion and no redraw: return
  now = common_now()                          error → status=1 → close
  elapsed = 0 on first frame, otherwise now - last_frame_time
  last_frame_time = now; clamp elapsed into [0, 0.05]
  if motion: save player → player_update → compare → mark dirty
  if dirty: screen_clear → screen_render → clear dirty flag
```

At 2.5 world units/second, a clamped step travels at most 0.125 units before collision. Rotation is 1.8 radians/second. Idle frames avoid redraw work, though the MiniLibX loop still runs.

The timestamp is not updated on the early idle return. The first update after a long idle can use the maximum 0.05-second step. Timing uses gettimeofday, so negative deltas are clamped.

SPEAKER NOTE Explain both why time scaling avoids frame-rate-dependent speed and why the cap bounds large pauses. The cap does not provide fixed-step simulation.

Movement, collision and rotation

```
forward = forward_key - backward_key
strafe  = left_key - right_key
normalize(forward, strafe)

move_x = (dir_x*forward + dir_y*strafe)
         * 2.5 * elapsed
move_y = (dir_y*forward - dir_x*strafe)
         * 2.5 * elapsed
```

- Normalize input so diagonal movement has the same total speed as moving straight.
- Test four corners at (x ± 0.2, y ± 0.2). Outside bounds or anything other than floor blocks movement.
- Try the X move first, then Y using the possibly updated X. This allows sliding along a wall.
- Move before turning. Rotate direction and camera plane by the same 2D matrix; left uses a negative angle and right a positive angle.

Rotation: `x′ = x cos(a) − y sin(a)`, `y′ = old_x sin(a) + y cos(a)`. Saving old_x avoids using an already changed value.

SPEAKER NOTE Collision is a four-corner square footprint, not an exact circular collision test. Opposite forward/back keys cancel.

Camera rays: direction plus a screen offset

ray index i → **camera_x in [-1,1]** → **dir + plane × camera_x** → **DDA hit**

```
camera_x = 2.0 * i / (300 - 1.0) - 1.0
ray_x = player.dir_x + player.plane_x * camera_x
ray_y = player.dir_y + player.plane_y * camera_x
```

- 300 rays cover a 1024-pixel view. Ray i draws pixels from `i*width/300` up to `(i+1)*width/300`, excluding the end.
- For an east-facing player, left edge ray = (1,-0.6), right edge = (1,0.6). With 300 samples, there is no exact center sample; rays 149 and 150 straddle it.
- Camera rays are deliberately unnormalized. Their DDA parameter is depth along the unit forward direction, which supplies the projection distance without an extra cosine correction.

SPEAKER NOTE Derivation: dir dot plane = 0 and |dir| = 1, so dir dot ray = 1. For hit = pos + t*ray, forward depth is t.

DDA: visit grid boundaries in order

```
_raycast_setup:
  map = floor(player position)
  delta_x = abs(1/ray_x)
  delta_y = abs(1/ray_y)
  step = sign of ray components
  side = time to first X/Y boundary

_raycast_advance:
  choose smaller side time
  record time, add delta, step one cell
```

- A zero ray component uses 1e30 as the unreachable boundary distance.
- Crossing X enters a vertical grid boundary (RAYSIDE_X); crossing Y enters a horizontal boundary (RAYSIDE_Y). Equal times take Y.
- After each step, reject out-of-bounds access; floor continues, any other cell records a hit.
- Stop at the first hit or after width + height steps. Hit includes world x/y, side, and the crossing parameter.

The ray moves monotonically through the grid. Choosing the next closest boundary avoids tiny fixed-distance sampling and does not skip a wall cell along the traversed path.

SPEAKER NOTE Do not confuse delta with distance to the first boundary: delta is the spacing between later boundaries on the same axis.

A worked ray from the supplied map

```
maps/001.cub, spawn (3.5,2.5), E
Actual camera sample i = 149:
  camera_x = -1/299
  ray = (1, -0.6/299)
  delta_x = 1; delta_y ≈ 498.333
  side_x = 0.5; side_y ≈ 249.167
```

Boundary time	Entered cell	Result
0.5	(4,2)	Floor → continue
1.5	(5,2)	Floor → continue
2.5	(6,2)	Floor → continue
3.5	(7,2)	Wall → hit

Hit ≈ (7.0, 2.49298), side X, depth 3.5. Positive ray_x chooses the west-facing wall texture. Projected height = int(768 / 3.5) = 219; top = 275. This sample fills screen columns 508–511.

For comparison, a conceptual exact center ray (1,0) hits (7,2.5). It illustrates the geometry but is not one of the 300 samples.

SPEAKER NOTE Walk the table by hand. This ties the player state, DDA time, texture direction, and screen strip into a single concrete example.

A hit becomes a textured wall strip

texture_for_hit → **column geometry** → **texture coordinates** → **copy pixels**

Calculation	Meaning
depth = max(hit.distance, 0.05)	Keep projection finite near a wall.
height = int(frame.height / depth)	Near walls are tall; distant walls are short.
top = H/2 − height/2; start/end clipped	Center the wall vertically, restrict writes to the screen.
offset = fract(hit.y) for X; fract(hit.x) for Y	Position along the struck wall; tex_x = int(offset * texture.width).
Flip tex_x if (X and ray_x>0) or (Y and ray_y<0)	Keep directional texture orientation consistent.
tex_y = int((y − top) * texture.height / height)	Use the unclipped top to preserve the crop when the wall extends off screen.

SPEAKER NOTE First paint ceiling and floor for each screen column, then overwrite the wall span. Texture lookup is nearest-neighbor with no lighting pass.

Which texture is visible, and where pixels go

Hit side	Ray sign	Texture face
X	ray_x > 0	WE
X	ray_x < 0	EA
Y	ray_y > 0	NO
Y	ray_y < 0	SO

```
pixel address = image.addr
              + y * image.stride
              + x * (image.bpp / 8)

32-bit color copy:
*(unsigned int *)dst = color;
```

- Names describe the wall face, not the direction the player is looking. Moving a ray east hits a west-facing surface.
- stride is bytes per row and may include padding; width × 4 must not replace it blindly.
- render_px clips to image bounds. The wall renderer uses its own already-clipped loops and direct buffer writes.
- Textures explicitly require 32 bpp. Frame writes also assume 32-bit pixels; frame setup stores its format but does not validate bpp or convert endian.

SPEAKER NOTE Treat pixel format support as a current implementation assumption. The endian field is stored but is not used to transform colors.

Screen composition and the minimap

```
screen_render
 ├─ V_3D: render_3d
 │        → _screen_overlay viewport
 ├─ V_MINIMAP: full-frame viewport
 ├─ render_map_view
 │   ├─ _render_map → each cell
 │   ├─ render_fov → 300 DDA rays
 │   └─ _render_player
 └─ mlx_put_image_to_window
```

- Overlay preserves map aspect ratio inside a 256 × 256 limit, at top right with a 16-pixel margin.
- Full-map mode uses the entire 1024 × 768 viewport; independent X/Y scaling may stretch map cells.
- World-to-map conversion: viewport origin + world coordinate × viewport size / grid dimension.
- Draw order matters: cells first, orange FOV rays second, green player marker last. Border cells use a separate border fill when large enough.

The 3D pass casts 300 rays, then the minimap casts another 300. Full-map mode casts only the minimap set. Hits are recomputed rather than shared between passes.

SPEAKER NOTE Use the minimap as a debugging view: it displays the same camera and DDA calculations used by the 3D renderer.

Drawing primitives and complexity

Function / stage	How it works	Cost
render_px	Bounds check, calculate byte address, write one color	O(1)
render_rect	Half-open X and Y loops call render_px	O(rectangle area)
render_line	Bresenham error accumulator; step X/Y toward endpoint	O(max(|dx|,|dy|))
Map validation / padding	Inspect or copy the rectangular grid	O(map width × height)
Appending map rows	Reallocate pointer array for every row	O(height²) pointer copies overall
Ray traversal	Each ray crosses at most width + height cells	O(rays × (width + height))
3D raster + minimap	Fill frame pixels, map cells, and FOV lines	O(frame area + grid area + ray traversal + line pixels)

Memory: grid O(width × height), four textures plus frame buffers, and bounded per-ray state. Dynamic line storage grows geometrically. Idle frames skip rendering; this is not a sleeping or frame-limited event loop.

SPEAKER NOTE Discuss optimization from measurements: caching minimap ray hits could remove duplicate traversal, but it is not implemented here.

Failures, closing, and resource release

Parse/setup failure OR close request → **leave setup / loop** → **runtime_cleanup** → **return status**

Path	Behavior
Bad arguments	Print Error + usage to stderr; return 1 without allocating game resources.
Read / map validation failure	Free local parser map, print Invalid map file; runtime setup cleans zero/partial game.
Graphics setup failure	Print the graphics/texture error; cleanup destroys only resources already created.
Frame time or rendering failure	Set exit_status to 1 through common_error, call runtime_close.
Escape / window close	Leave status 0; request loop end; runtime_run cleans afterward.

Cleanup order: texture images → frame image → window → display and context → grid rows / grid / texture paths. Graphics handles and map are reset. The context must outlive destruction of its images and window.

SPEAKER NOTE Explain why zero-initialization makes partial cleanup safe, and why parser transfer clears the old owner.

Three end-to-end traces to rehearse

Trigger	Exact journey
Hold W	MLX key event → hook_press → common_key_id → keys[GK_FORWARD]=1 → game_frame → player_update → _player_move → corner collision checks → changed state → screen_clear → screen_render → 3D + map → image blit.
Press M	hook_press checks GK_VIEW latch → screen_toggle_view → view changes + dirty=1 → game_frame → screen_render chooses full-map viewport → render_map_view → image blit. Release clears latch.
Malformed RGB	parser_next_line → parser_process_line → parser_parse_config → _parser_parse_color_config → parser_parse_color fails → stop file processing → close fd → parser_cleanup(local map) → common_error → runtime_cleanup(game) → main returns 1.

For each trace, name the data that changes, which function owns it, and what value signals success or failure.

SPEAKER NOTE An evaluator can enter the code at any function. Use these traces to reconnect it to the larger lifecycle.

Build, tests and a practical demonstration

```
make
./cub3D maps/001.cub
bash tests/test_maps.sh

Controls:
W/S or up/down: forward/back
A/D: strafe     left/right: turn
M: map view    Escape: exit
```

- Verified for this document: build completed; all 18 valid fixtures accepted; all 27 invalid fixtures rejected.
- The test script builds a parser-only checker. --textures checks path readability with access; it does not initialize MLX or decode XPM images.
- Tests cover required fields, RGB errors, illegal characters, spawn count, enclosure, ordering, extensions, and accepted disconnected regions.
- For a live presentation: walk, strafe into a wall, rotate, toggle M, release focus, then exit. These graphics interactions were not exercised by the parser tests.

Executable name is case-sensitive: `cub3D`, as defined in Makefile. The README example uses a different case. Vendored MLX emits a build warning; project sources compile with -Wall -Wextra -Werror.

SPEAKER NOTE Report parser checks as parser checks. Do not claim automated coverage of drawing, key events, or graphical resource teardown.

Questions you should be ready to answer

Question	Answer to rehearse
Why no fish-eye correction?	Unnormalized camera ray has dot(dir,ray)=1, so its DDA parameter is forward depth.
Why does diagonal motion not speed up?	Forward/strafe inputs are normalized before scaling by speed and elapsed time.
Why do short rows use spaces?	Missing cells must remain void so enclosure checks detect openings.
Why copy the map into g and clear data.map?	Ownership transfers without duplicating allocations or freeing them twice.
Why can the player slide along a wall?	X and Y moves are tested independently against the four-corner footprint.
Why rotate both direction and plane?	They must keep the same camera orientation, perpendicularity and plane scale.
Why does closing not immediately free g?	Callbacks request loop termination; runtime_run frees resources after control returns.
Is the map validated with flood fill?	No. Every walkable cell is checked for cardinal enclosure; disconnected enclosed rooms pass.

SPEAKER NOTE Answer from the actual code. If asked about alternatives, distinguish current behavior from possible future changes.

Math foundations: vectors, cells and units

```
p = (pos_x, pos_y)       world position
d = (dir_x, dir_y)       unit forward vector
c = (plane_x, plane_y)   camera-plane vector
r = d + u*c              ray direction
q(t) = p + t*r           point on that ray

|v| = sqrt(v_x² + v_y²)
a dot b = a_x*b_x + a_y*b_y
```

- One map tile is one world unit wide. World coordinates are continuous; grid indices are integers.
- Cell (i,j) occupies [i,i+1) × [j,j+1). floor maps a position to that cell; center is (i+0.5,j+0.5).
- floor(-0.2) = -1, while a C integer cast gives 0. Collision uses floor before rejecting negative indices.
- Distance, ray parameter, and pixel counts are different quantities. They agree only under specific scaling assumptions.

SPEAKER NOTE Define symbols before deriving anything. Here c is a vector, not the C color field; u is a dimensionless screen coordinate.

Why diagonal normalization and strafe signs work

```
a = forward_key - backward_key; b = left_key - right_key
L = sqrt(a² + b²); if L == 0, no displacement
(a,b) := (a/L, b/L)
left unit vector l = (d_y, -d_x)
velocity v = PLAYER_SPEED * (a*d + b*l)
```

Because |d| = |l| = 1 and d dot l = 0, $|a*d + b*l|^2 = a^2 + b^2 = 1$. Thus |v| = 2.5 whether one key or a diagonal combination is held.

Example: facing east d=(1,0), W+A gives (a,b)=(1/√2,1/√2). For dt=0.02 s: displacement ≈ (0.035355,−0.035355), whose length is 0.05. Without normalization its length would be 0.070711.

The Y sign follows downward-positive coordinates: left of east points upward, l=(0,−1). Floating-point rotation can slightly change the unit-length invariant over long runs.

SPEAKER NOTE Show the Pythagorean calculation. The input vector is normalized; the code does not renormalize the direction after each turn.

Rotation: where sine and cosine enter

```
R(a) = [ cos(a)  -sin(a) ]
       [ sin(a)   cos(a) ]

v_new = R(a) * v_old
a = turn_sign * 1.8 * dt

R(a)^T * R(a) = I
|R(a)*v| = |v|
(R(a)*d) dot (R(a)*c) = d dot c
```

- Radian measure: a full turn is 2π radians = 360°. 1.8 rad/s is approximately 103.13°/s.
- At dt=0.05, a right turn is 0.09 rad ≈ 5.157°. East becomes (cos 0.09, sin 0.09) ≈ (0.995953,0.089879).
- The plane (0,0.6) becomes approximately (−0.053927,0.597572). Length remains 0.6 and it stays perpendicular to direction.
- In screen/world coordinates with Y downward, positive angle turns clockwise. Save the old X value before changing Y.

SPEAKER NOTE Orthogonality preservation follows from R transpose times R equaling identity. Both vectors must use the same matrix.

Time integration and the collision footprint

Quantity	Formula / interpretation
Time conversion	seconds = tv_sec + tv_usec / 1,000,000; dt = clamp(now − previous, 0, 0.05). First active frame uses dt=0.
Translation and turn	Δp = velocity × dt; Δangle = angular_speed × dt. Both are explicit per-frame updates.
Collision footprint	Candidate square [x−0.2,x+0.2] × [y−0.2,y+0.2]; query its four corners with floor.
Maximum update	Translation ≤ 2.5 × 0.05 = 0.125 world units when |d|≈1; a one-direction turn ≤ 0.09 rad.
Sliding	Accept X at current Y, then accept Y at the resulting X. This is axis-separated endpoint testing, not swept collision.
Example at wall x=7	A center at x=6.79 has right edge 6.99: floor cell. At x=6.81 its right edge 7.01 reaches wall cell 7: blocked.

The square is only 0.4 units wide, less than a cell, so its corners cover the cells touched by its footprint. These constants keep steps below a tile width; this is not a general continuous-collision proof for arbitrary speeds, radii or geometry.

SPEAKER NOTE A long pause is capped, so the simulation discards part of the elapsed time rather than catching up through multiple fixed steps.

Camera plane and the field-of-view triangle

```
c = 0.6 * (-d_y, d_x)
d dot c = 0; |d| = 1; |c| = 0.6
ray at edge = d ± c
tan(FOV_x / 2) = opposite / adjacent = |c| / |d|
FOV_x = 2 * atan(0.6) ≈ 1.080839 rad ≈ 61.9275°
```

The conceptual camera plane is one unit forward from p and extends 0.6 to either side. Each ray goes from p through a point on that plane. Increasing plane scale widens the view; increasing direction length without scaling the plane narrows it.

For N rays, u_i = 2i/(N−1)−1 includes both edges. Equal steps in u are equal camera-plane steps, not equal angular steps: angle_i = atan(0.6u_i). For N=1, this implementation uses u=0.

SPEAKER NOTE Use a right triangle with forward side 1 and lateral side 0.6. Do not describe the samples as evenly spaced angles.

Deriving DDA increments and first boundaries

```
Ray equation: q_x(t) = p_x + t*r_x
At vertical boundary x=b: t = (b - p_x) / r_x
Consecutive vertical boundaries differ by one cell:
    delta_x = |1 / r_x|; likewise delta_y = |1 / r_y|

m_x = floor(p_x)
if r_x > 0: first boundary b_x = m_x + 1
if r_x < 0: first boundary b_x = m_x
side_x = |b_x - p_x| * delta_x   (and likewise for Y)
```

The code compresses the branch into b_x = map_x + (step_x > 0), where the comparison contributes 1 or 0. It stores the chosen boundary time before adding delta, so hit.distance already equals t and needs no “subtract delta” correction.

If r_x=0, there is no vertical crossing. Code uses a finite 1e30 sentinel rather than dividing by zero. It is a practical approximation for ordinary map dimensions.

SPEAKER NOTE delta is a change in ray parameter t. It is Euclidean distance only for a unit-length ray, which these camera rays generally are not.

An oblique DDA trace, including the tie rule

Take p=(3.5,2.5), east-facing left-edge ray r=(1,−0.6). Then delta=(1,1.666667), step=(+1,−1), initial side=(0.5,0.833333). In maps/001.cub:

Choose	Time t	Entered cell	Next side times / result
X	0.5	(4,2)	side_x=1.5, side_y≈0.833333; floor
Y	≈0.833333	(4,1)	side_x=1.5, side_y=2.5; floor
X	1.5	(5,1)	side_x=2.5, side_y=2.5; floor
Y on equality	2.5	(5,0)	Top wall: hit at (6.0,1.0); RAYSIDE_Y; SO texture

At a grid corner, several cells touch the same point. The strict side_x < side_y comparison chooses Y on equality; it does not test every corner-adjacent cell. Corner hits can select a different face if floating-point rounding changes a near-tie.

The width+height bound follows because every iteration advances X or Y monotonically by one cell; a ray must leave a finite rectangle within that many crossings if no hit stops it.

SPEAKER NOTE This trace complements the nearly horizontal ray: it exercises both axes and shows the exact boundary tie convention.

Depth, Euclidean length, and the fish-eye proof

```
r = d + u*c; |d|=1; d dot c=0
hit - p = t*r
forward depth D = (hit-p) dot d
                = t * (d dot d + u*c dot d) = t
Euclidean ray length E = |t*r| = t*|r|
cos(theta) = (r dot d)/(|r|*|d|) = 1/|r|
Therefore: E*cos(theta) = t = D
```

For edge ray r=(1,−0.6), |r|=√1.36≈1.166190. At t=2.5, Euclidean length ≈2.915476 while forward depth is 2.5. Projecting with Euclidean length would shrink this edge wall by factor 1/1.166190≈0.857493.

The stored hit.distance is already t, so no explicit cosine multiplication is needed. Normalizing r without changing the distance calculation would invalidate that shortcut. The proof assumes exact camera invariants; repeated floating-point rotations preserve them approximately.

SPEAKER NOTE A straight wall at constant forward depth should have constant projected height. Using longer edge-ray lengths makes it appear curved.

Projection: similar triangles and the chosen scale

```
Similar triangles: projected_height / focal_pixels = wall_height / D
General form: projected_height = focal_pixels * wall_height / D
Implementation: height = int(screen_height / max(D, 0.05))
This uses focal_pixels * wall_height = screen_height = 768.
```

For a unit-height wall and a square-pixel camera calibrated to horizontal FOV, $f_x = (W/2)/\tan(FOV_x/2) = 1024/(2\times0.6) \approx 853.333$ pixels. This code uses 768 for the vertical wall scale, so its walls are $768/853.333 = 0.9$ times that particular unit-wall calibration.

That is an independent vertical scale choice, not a DDA error or a fish-eye effect. Equivalently, one could interpret the walls as height 0.9 with the horizontal focal calibration. The code does not explicitly define a world-space wall-height parameter.

The 0.05 depth clamp caps projected height at 15,360 pixels. Very distant walls can truncate to height 0; the empty drawing span then avoids a tex_y division by zero.

SPEAKER NOTE Do not claim that screen_height/D follows uniquely from plane scale 0.6. Similar triangles explain inverse depth, but the focal/height scale is an implementation choice.

Screen strips, clipping and integer arithmetic

Stage	Exact arithmetic and consequence
Strip partition	x_start=floor(iW/N), x_end=floor((i+1)W/N). End-exclusive strips share boundaries; first starts at 0 and final ends at W.
1024 pixels / 300 rays	1024=300×3+124: 176 strips are 3 pixels wide and 124 are 4 pixels wide. Each strip reuses one texture column.
Vertical centering	top = H/2 − height/2 with C integer division. H=768 and height=219 give top=384−109=275; floating-point centering would differ.
Visible interval	start=max(top,0); end=min(top+height,H). Draw start ≤ y < end.
Near-wall example	D=0.5 → height=1536, top=−384, visible y=0..767. The screen shows the central section of the tall wall.
Sampling caveat	u includes camera-plane endpoints, while strips cover intervals of pixels. The mapping is a finite approximation, not one ray through each pixel center.

SPEAKER NOTE Explain half-open intervals once and use that convention consistently for rectangles, screen strips and texture spans.

Texture coordinates: fractional position and crop

```
s = hit.y for X-side hits; hit.x for Y-side hits
u = s - floor(s)                        so 0 ≤ u < 1
tex_x = floor(u * texture_width)
if face orientation needs reversal: tex_x = width - 1 - tex_x
tex_y = int((y - top) * texture_height / projected_height)
```

Using the worked ray i=149 and the actual 16×16 west texture: hit.y≈2.492976589, u≈0.492976589, raw tex_x=7, flipped tex_x=8. At screen y=384 with top=275 and height=219, tex_y=floor(109×16/219)=7.

Near-wall example: height=1536, top=−384, texture height=16. Visible top row y=0 samples tex_y=4; y=767 samples 11. Using clipped start=0 as the origin would incorrectly stretch the full texture onto the screen.

Because 0≤u<1, tex_x stays in [0,width−1]. For a nonempty wall span, 0≤y−top<height, keeping tex_y in [0,texture_height−1]. Tiny floating-point differences near a texel boundary may change the selected texel.

SPEAKER NOTE The wall coordinate uses the axis along the wall: Y on vertical walls, X on horizontal walls. Sampling uses nearest-neighbor truncation.

Minimap transform and pixel/color arithmetic

```
overlay scale = min(256/map_W, 256/map_H)
v_W = int(map_W * scale)
v_H = int(map_H * scale)
v_x = frame_W - v_W - 16; v_y = 16

px = v_x + int(world_x * v_W / map_W)
py = v_y + int(world_y * v_H / map_H)
```

- For the 8×7 sample map: overlay 256×224 at (752,16). Spawn (3.5,2.5) maps to pixel (864,96).
- Full-map mode uses 1024×768, so a tile spans 128 pixels horizontally and about 109.714 vertically. Integer boundaries can make adjacent tile heights differ by one pixel.
- RGB packing is R×65536+G×256+B. A channel uses eight bits; 50,50,50 gives 3,289,650 = 0x323232.
- At 32 bpp with stride=4096, pixel (10,20) is 20×4096+10×4 = 81,960 bytes from addr. Actual stride comes from MLX.

SPEAKER NOTE Viewport fitting avoids clipping the map. Independent viewport conversion in full-map mode stretches its proportions.

Bresenham: integer error instead of slope division

```
dx=|to.x-from.x|; dy=|to.y-from.y|
sx=sign(to.x-from.x); sy=sign(to.y-from.y)
error=dx-dy
Draw current point; stop if at endpoint. Otherwise:
  twice = 2*error                          save once
  if twice > -dy: error -= dy; x += sx
  if twice <  dx: error += dx; y += sy
```

The ideal segment has a fixed ratio of horizontal to vertical progress. The error accumulator tracks the imbalance in integer units; subtracting dy accounts for an X step, adding dx accounts for a Y step. Both tests can pass in one iteration.

Example from (0,0) to (5,3): dx=5, dy=3, initial error=2. Drawn points are (0,0), (1,1), (2,1), (3,2), (4,2), (5,3). After the first step, error=4; the next iteration steps only X.

The saved twice value must feed both decisions. Endpoint equality is checked before stepping, so a zero-length segment draws once. No dy/dx division is needed, including for vertical lines.

SPEAKER NOTE This is the minimap FOV line rasterizer. Its error measures a discrete line decision, not the world distance to a wall.

common.c

`int common_key_id(int keycode)`

Map X11 keycodes to logical movement/view IDs; unknown keys return -1. W/up and S/down each share an ID.

Called by: hook_press, hook_release
Calls: No project functions

source/common.c:15 • public function

`double common_now(void)`

Read gettimeofday and return seconds as a double, or -1.0 on failure.

Called by: game_frame
Calls: No project functions | Library/system: gettimeofday

source/common.c:34 • public function

`int common_error(char *message)`

Write Error and the supplied message to stderr; return 1 for a caller to propagate or store.

Called by: _game_render, game_frame, parser_load_map, runtime_args, _setup_mlx, _setup_frame, _texture_load
Calls: No project functions | Library/system: ft_putendl_fd

source/common.c:43 • public function

SPEAKER NOTE Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

game.c

`static int _game_has_motion(t_game *g)`

Return whether any of the six movement/turn booleans is set; view toggle is excluded.

Called by: game_frame
Calls: No project functions

source/game.c:15 • file-local helper

`static int _game_player_changed(t_player *old, t_player *current)`

Compare all position, direction and plane components; return whether any value changed.

Called by: _game_update_player
Calls: No project functions

source/game.c:22 • file-local helper

`static void _game_update_player(t_game *g, double elapsed)`

Snapshot the player, update movement/rotation, and set needs_redraw only if the player changed.

Called by: game_frame
Calls: player_update, _game_player_changed

source/game.c:30 • file-local helper

`static int _game_render(t_game *g)`

Skip a clean frame; otherwise clear and render. On failure set exit_status and request loop end; on success clear needs_redraw.

Called by: game_frame
Calls: screen_clear, screen_render, common_error, runtime_close

source/game.c:40 • file-local helper

`int game_frame(t_game *g)`

Loop callback: early-out when idle, obtain/clamp elapsed time, optionally update player, then render if dirty. Returns callback status.

Called by: MLX loop callback (registered in _setup_hook)
Calls: _game_has_motion, common_now, common_error, runtime_close, _game_update_player, _game_render

source/game.c:53 • public function

SPEAKER NOTE Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

hook.c

`int hook_press(int keycode, t_game *g)`

Escape requests close; M toggles once per latched press; other recognized keys set logical held-state and request redraw on transition.

Called by: MLX key-press event
Calls: runtime_close, common_key_id, screen_toggle_view

source/hook.c:15 • public function

`int hook_release(int keycode, t_game *g)`

Clear the mapped logical key; request redraw when the key had been held. Unknown keycodes have no effect.

Called by: MLX key-release event
Calls: common_key_id

source/hook.c:39 • public function

`int hook_focus_out(t_game *g)`

Clear every held key and mark dirty if any key was active, preventing motion from sticking after focus loss.

Called by: MLX focus-out event
Calls: No project functions

source/hook.c:53 • public function

`int hook_expose(t_game *g)`

Mark the frame dirty when the window is exposed; actual composition occurs in the frame callback.

Called by: MLX expose event
Calls: No project functions

source/hook.c:67 • public function

SPEAKER NOTE Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

main.c

`int main(int argc, char **argv)`

Validate arguments, initialize the local game, then run the MLX loop. Argument/setup failures return 1.

Called by: C runtime entry
Calls: runtime_args, runtime_setup, runtime_run

source/main.c:15 • public function

SPEAKER NOTE Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

parser.c

`static void _parser_apply_player(t_game *g, t_parser_data *data)`

Convert N/S/E/W to unit direction, place the player at cell center, and construct a perpendicular plane scaled by 0.6.

Called by: _parser_transfer
Calls: No project functions

source/parser/parser.c:15 • file-local helper

`static void _parser_transfer(t_game *g, t_parser_data *data)`

Move the local map into g, zero the previous owner, replace the spawn with floor, and apply player coordinates/orientation.

Called by: parser_load_map
Calls: _parser_apply_player

source/parser/parser.c:38 • file-local helper

`void parser_cleanup(t_map *map)`

Free all owned grid rows, the pointer array, and four texture path strings; reset the map. NULL map is accepted.

Called by: parser_load_map, runtime_cleanup, tests/map_check.c:main
Calls: No project functions | Library/system: free

source/parser/parser.c:46 • public function

`int parser_load_map(t_game *g, char *map_path)`

Build local parser state, read/validate, then transfer ownership to g. Free local data and report an error on failure. Returns 0 success / 1 failure.

Called by: runtime_setup, tests/map_check.c:main
Calls: common_error, parser_read_file, parser_validate_map, parser_cleanup, _parser_transfer

source/parser/parser.c:62 • public function

SPEAKER NOTE Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

parser_cells.c

`int parser_is_player(char cell)`

Return true only for N, S, E or W spawn markers.

Called by: parser_is_walkable, _parser_inspect_map_line
Calls: No project functions

source/parser/parser_cells.c:15 • public function

`int parser_is_walkable(char cell)`

Return true for floor or a player marker; spaces and walls are not walkable.

Called by: _parser_is_map_start, _parser_is_closed_cell, _parser_validate_closed_cells
Calls: parser_is_player

source/parser/parser_cells.c:20 • public function

`char parser_map_at(t_parser_data *data, int x, int y)`

Return a grid cell, or space for a missing map or coordinates outside its rectangular dimensions.

Called by: _parser_is_closed_cell, _parser_validate_closed_cells
Calls: No project functions

source/parser/parser_cells.c:25 • public function

SPEAKER NOTE Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

parser_color.c

`static void _parser_skip_space(const char **value)`

Advance a borrowed string cursor over spaces and tabs; allocate nothing.

Called by: _parser_parse_rgb_value
Calls: No project functions

source/parser/parser_color.c:15 • file-local helper

`static int _parser_parse_rgb_value(const char **value, int *component)`

Parse one unsigned decimal component, allowing surrounding spaces/tabs; reject absent digits or values above 255; advance cursor.

Called by: _parser_parse_rgb
Calls: _parser_skip_space

source/parser/parser_color.c:21 • file-local helper

`static int _parser_parse_separator(const char **value)`

Require a comma at the cursor, then consume it. Return 1 success or 0 missing separator.

Called by: _parser_parse_rgb
Calls: No project functions

source/parser/parser_color.c:41 • file-local helper

`static int _parser_parse_rgb(const char *value, int *red, int *green, int *blue)`

Parse three components separated by two commas; require end of string after the third. Return a success boolean.

Called by: parser_parse_color
Calls: _parser_parse_rgb_value, _parser_parse_separator

source/parser/parser_color.c:49 • file-local helper

`int parser_parse_color(const char *value, int *color)`

Validate input/output pointers, parse RGB, then pack 0xRRGGBB into the destination. Return 1 on success, 0 on failure.

Called by: _parser_parse_color_config
Calls: _parser_parse_rgb

source/parser/parser_color.c:63 • public function

> **SPEAKER NOTE** Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

parser_config.c

`static unsigned int _parser_texture_field(const char *line)`

Recognize a NO/SO/WE/EA two-character prefix and return its field bit; otherwise return 0. Whitespace is checked later.

Called by: _parser_parse_texture_config, parser_parse_config
Calls: No project functions | Library/system: ft_strncmp

source/parser/parser_config.c:15 • file-local helper

`static char **_parser_texture_slot(t_parser_data *data, unsigned int field)`

Return the address of the texture-path slot for a recognized field bit, or NULL.

Called by: _parser_parse_texture_config
Calls: No project functions

source/parser/parser_config.c:28 • file-local helper

`static int _parser_parse_texture_config(t_parser_data *data, char *line)`

Require identifier whitespace, reject duplicates, allocate a trimmed nonempty path, store it and set its field bit. Return boolean success.

Called by: parser_parse_config
Calls: _parser_texture_field, _parser_texture_slot | Library/system: ft_strtrim, free

source/parser/parser_config.c:41 • file-local helper

`static int _parser_parse_color_config(t_parser_data *data, char *line)`

Select F or C destination, enforce whitespace and uniqueness, parse RGB and set the corresponding field bit. Return boolean success.

Called by: parser_parse_config
Calls: parser_parse_color

source/parser/parser_config.c:64 • file-local helper

`int parser_parse_config(t_parser_data *data, char *line)`

Reject empty input; dispatch texture prefixes or F/C to their config parsers. Unknown identifiers return 0.

Called by: _parser_process_content
Calls: _parser_texture_field, _parser_parse_texture_config, _parser_parse_color_config

source/parser/parser_config.c:84 • public function

> **SPEAKER NOTE** Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

parser_grid.c

`static void _parser_free_grid(char **grid, int height)`

Free up to height row pointers and the grid array. Handles NULL grid and zeroed row pointers.

Called by: _parser_copy_grid, parser_rectangularize
Calls: No project functions | Library/system: free

source/parser/parser_grid.c:15 • file-local helper

`static char *_parser_pad_row(const char *row, int width)`

Allocate width+1 bytes, copy a row, pad the rest with spaces, and append NUL. Return the new row or NULL.

Called by: _parser_copy_grid
Calls: No project functions | Library/system: malloc

source/parser/parser_grid.c:25 • file-local helper

`static char **_parser_copy_grid(t_map *map)`

Allocate a zeroed pointer array and padded copies of all rows. Clean the new partial grid if a copy fails.

Called by: parser_rectangularize
Calls: _parser_pad_row, _parser_free_grid | Library/system: ft_calloc

source/parser/parser_grid.c:45 • file-local helper

`int parser_rectangularize(t_map *map)`

Create the complete padded grid before freeing the original grid, then swap it in. Return 1 success / 0 allocation failure.

Called by: parser_read_file
Calls: _parser_copy_grid, _parser_free_grid

source/parser/parser_grid.c:67 • public function

> **SPEAKER NOTE** Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

parser_line_reader.c

`static int _parser_read_byte(t_parser_reader *reader, char *byte)`

Refill the 4096-byte buffer when consumed, retry EINTR, and output one byte. Return 1 byte / 0 EOF / -1 error.

Called by: _parser_collect_line
Calls: No project functions | Library/system: read

source/parser/parser_line_reader.c:16 • file-local helper

`static int _parser_append_byte(t_parser_line *line, char byte)`

Append to a growing line; start at capacity 128 and double with overflow protection. Return 1 success / 0 failure.

Called by: _parser_collect_line
Calls: No project functions | Library/system: malloc, ft_memcpy, free

source/parser/parser_line_reader.c:34 • file-local helper

`static int _parser_collect_line(t_parser_reader *reader, t_parser_line *line)`

Collect non-newline bytes, reject embedded NUL, append a terminator, and support final unterminated lines. Return 1 line / 0 EOF / -1 error.

Called by: parser_next_line
Calls: _parser_read_byte, _parser_append_byte

source/parser/parser_line_reader.c:59 • file-local helper

`int parser_next_line(t_parser_reader *reader, char **text)`

Initialize temporary line storage; transfer its allocation to the caller only on status 1, otherwise free it. Set output NULL before reading.

Called by: _parser_read_lines
Calls: _parser_collect_line | Library/system: free

source/parser/parser_line_reader.c:78 • public function

SPEAKER NOTE Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

parser_lines.c

`static int _parser_is_map_start(const char *line)`

Exclude texture prefixes; classify an initial wall or walkable marker as the start of map content.

Called by: _parser_process_content
Calls: parser_is_walkable

source/parser/parser_lines.c:15 • file-local helper

`static int _parser_process_content(t_parser_data *data, char *line, char *trimmed)`

Drive before/in/after-map state. Return 0 rejected, 1 accepted without ownership transfer, or 2 accepted with original line stored in grid.

Called by: parser_process_line
Calls: _parser_is_map_start, parser_store_map_line, parser_parse_config

source/parser/parser_lines.c:24 • file-local helper

`int parser_process_line(t_parser_data *data, char *line)`

Trim a classification copy, process content, free the copy, and free the original unless stored. Return accepted/rejected boolean.

Called by: _parser_read_lines
Calls: _parser_process_content | Library/system: ft_strtrim, free

source/parser/parser_lines.c:44 • public function

SPEAKER NOTE Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

parser_map_lines.c

`static char **_parser_resize_grid(t_parser_data *data)`

Allocate height+2 row pointers, copy existing entries and terminator, then free the old pointer array. Row strings retain ownership.

Called by: parser_store_map_line
Calls: No project functions | Library/system: malloc, ft_memcpy, free

source/parser/parser_map_lines.c:15 • file-local helper

`int parser_store_map_line(t_parser_data *data, char *line)`

Check integer/allocation limits; append the owned original row, update height/max width, and maintain NULL termination. Return boolean success.

Called by: _parser_process_content
Calls: _parser_resize_grid | Library/system: ft_strlen

source/parser/parser_map_lines.c:31 • public function

SPEAKER NOTE Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

parser_reader.c

`static int _parser_read_lines(t_parser_data *data, int fd)`

Initialize the reader and process lines until EOF. Reject reader errors or the first rejected line; return boolean success.

Called by: parser_read_file
Calls: parser_next_line, parser_process_line

source/parser/parser_reader.c:15 • file-local helper

`int parser_read_file(t_parser_data *data, const char *path)`

Validate path suffix, open/read/close, require nonempty dimensions, then rectangularize. Return 1 success / 0 failure.

Called by: parser_load_map
Calls: _parser_read_lines, parser_rectangularize | Library/system: ft_strlen, ft_strncmp, open, close

source/parser/parser_reader.c:33 • public function

SPEAKER NOTE Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

parser_validate.c

`static int _parser_is_closed_cell(t_parser_data *data, int x, int y)`

Require all four cardinal neighbors to be wall or walkable. Out-of-bounds and void neighbors fail.

Called by: _parser_validate_closed_cells
Calls: parser_map_at, parser_is_walkable

source/parser/parser_validate.c:15 • file-local helper

`static int _parser_inspect_map_line(t_parser_data *data, int y, int *players)`

Reject illegal characters; count and record player marker coordinates/direction. Return 1 valid row / 0 invalid character.

Called by: _parser_scan_cells
Calls: parser_is_player

source/parser/parser_validate.c:32 • file-local helper

`static int _parser_validate_closed_cells(t_parser_data *data)`

Inspect every rectangular cell; all walkable cells must satisfy cardinal enclosure. Does not perform reachability traversal.

Called by: parser_validate_map
Calls: parser_is_walkable, parser_map_at, _parser_is_closed_cell

source/parser/parser_validate.c:56 • file-local helper

`static int _parser_scan_cells(t_parser_data *data)`

Scan rows and aggregate player count; return -1 for invalid characters or the total number of markers.

Called by: parser_validate_map
Calls: _parser_inspect_map_line

source/parser/parser_validate.c:77 • file-local helper

`int parser_validate_map(t_parser_data *data)`

Require all six fields, a grid at least 3 by 3, exactly one player, legal cells and enclosure. Return boolean success.

Called by: parser_load_map
Calls: _parser_scan_cells, _parser_validate_closed_cells

source/parser/parser_validate.c:93 • public function

> **SPEAKER NOTE** Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

player.c

`static int _player_cell_blocked(t_game *g, double x, double y)`

Floor world coordinates to cell coordinates; treat outside-grid and every non-floor cell as blocked.

Called by: _player_position_valid
Calls: No project functions | Library/system: floor

source/player.c:15 • file-local helper

`static int _player_position_valid(t_game *g, double x, double y)`

Check the four square-footprint corners at offsets +/- PLAYER_RADIUS; return true only if all are unblocked.

Called by: _player_move
Calls: _player_cell_blocked

source/player.c:29 • file-local helper

`static void _player_move(t_game *g, double elapsed)`

Normalize forward/strafe inputs, calculate time-scaled displacement, test/commit X then Y independently for wall sliding.

Called by: player_update
Calls: _player_position_valid | Library/system: sqrt

source/player.c:40 • file-local helper

`static void _player_rotate(t_player *p, double angle)`

Apply the same sine/cosine rotation to direction and camera plane, preserving the camera relationship.

Called by: player_update
Calls: No project functions | Library/system: cos, sin

source/player.c:65 • file-local helper

`void player_update(t_game *g, double elapsed)`

Move using the current direction, then apply left and right rotations according to held keys and elapsed time.

Called by: _game_update_player
Calls: _player_move, _player_rotate

source/player.c:81 • public function

SPEAKER NOTE Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

raycast.c

`static void _raycast_setup(t_game *g, t_raycast *rc)`

Initialize current grid cell, reciprocal-component delta values, signed steps and first-boundary times for this ray.

Called by: raycast_cast
Calls: No project functions | Library/system: floor, fabs

source/raycast.c:15 • file-local helper

`static void _raycast_advance(t_raycast *rc)`

Choose the smaller side time (ties choose Y), record hit.distance, increment that time and step one cell; store crossed side.

Called by: raycast_cast
Calls: No project functions

source/raycast.c:37 • file-local helper

`static int _raycast_in_boundaries(t_game *g, t_raycast *rc)`

Return whether the stepped cell lies within the map before any grid lookup occurs.

Called by: raycast_cast
Calls: No project functions

source/raycast.c:55 • file-local helper

`static int _raycast_record_hit(t_game *g, t_raycast *rc)`

Continue on floor; otherwise record crossed side and world hit point pos + ray*distance. Return 1 for a hit.

Called by: raycast_cast
Calls: No project functions

source/raycast.c:61 • file-local helper

`int raycast_cast(t_game *g, t_raycast *rc)`

Reset hit, initialize DDA, and traverse at most width+height steps; return 1 for first non-floor hit, 0 for escape or step exhaustion.

Called by: _render_fov_ray, _render_3d_ray
Calls: _raycast_setup, _raycast_advance, _raycast_in_boundaries, _raycast_record_hit

source/raycast.c:71 • public function

> **SPEAKER NOTE** Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

raycast_camera.c

`void raycast_camera_ray(t_game *g, t_raycast *ray, int index)`

Zero a ray result and calculate dir + plane*camera_x for an indexed camera sample. Handles a single configured ray with camera_x=0.

Called by: _render_fov_ray, _render_3d_ray
Calls: No project functions

source/raycast_camera.c:15 • public function

SPEAKER NOTE Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

render_draw.c

`static void _render_line_step(t_point *p, t_line *line)`

Update Bresenham error and move the current point in X and/or Y using the saved doubled error.

Called by: render_line
Calls: No project functions

source/render/render_draw.c:15 • file-local helper

`void render_px(t_img *img, t_point p, int color)`

Clip coordinates to image bounds, compute address using stride and bpp, then write a 32-bit color.

Called by: render_line, render_rect
Calls: No project functions

source/render/render_draw.c:32 • public function

`void render_line(t_img *img, t_point from, t_point to, int color)`

Initialize integer Bresenham state and draw an inclusive segment, calling render_px for each point until the endpoint.

Called by: _render_fov_ray
Calls: render_px, _render_line_step | Library/system: ft_abs

source/render/render_draw.c:43 • public function

`void render_rect(t_img *img, t_point from, t_point to, int color)`

Fill a rectangle with half-open coordinate bounds using render_px; empty ranges perform no writes.

Called by: _render_map_cell, _render_player
Calls: render_px

source/render/render_draw.c:61 • public function

`t_point render_world_to_map(t_game *g, t_viewport *v, double x, double y)`

Scale world coordinates independently into viewport X/Y dimensions and add viewport origin. Return integer pixel coordinates.

Called by: _render_fov_ray, render_fov, _render_map_cell, _render_player
Calls: No project functions

source/render/render_draw.c:78 • public function

> **SPEAKER NOTE** Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

render_fov.c

`static void _render_fov_ray(t_game *g, t_viewport *v, int i, t_point from)`

Build/cast one camera ray; on a hit convert its world endpoint and draw an orange line from the player.

Called by: render_fov
Calls: raycast_camera_ray, raycast_cast, render_world_to_map, render_line

source/render/render_fov.c:15 • file-local helper

`void render_fov(t_game *g, t_viewport *v)`

Convert player position once, then draw all 300 camera rays through the minimap viewport.

Called by: render_map_view
Calls: render_world_to_map, _render_fov_ray

source/render/render_fov.c:28 • public function

SPEAKER NOTE Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

render_view.c

`static int _render_tile_color(char tile)`

Choose gray for walls, dark gray for floor, and black for void.

Called by: _render_map_cell
Calls: No project functions

source/render/render_view.c:15 • file-local helper

`static void _render_map_cell(t_game *g, t_viewport *v, t_point cell)`

Convert cell corners to screen coordinates, optionally draw a border, then fill the cell interior with its tile color.

Called by: _render_map
Calls: render_world_to_map, render_rect, _render_tile_color

source/render/render_view.c:24 • file-local helper

`static void _render_map(t_game *g, t_viewport *v)`

Visit every rectangular grid coordinate and draw its cell inside the given viewport.

Called by: render_map_view
Calls: _render_map_cell

source/render/render_view.c:43 • file-local helper

`static void _render_player(t_game *g, t_viewport *v)`

Map the player center to pixels; draw a green square sized from tile scale with a two-pixel minimum.

Called by: render_map_view
Calls: render_world_to_map, render_rect

source/render/render_view.c:60 • file-local helper

`void render_map_view(t_game *g, t_viewport *v)`

Compose map cells, FOV rays and player marker in that order into the frame buffer.

Called by: screen_render
Calls: _render_map, render_fov, _render_player

source/render/render_view.c:79 • public function

> **SPEAKER NOTE** Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

render_wall.c

`static void _render_background(t_game *g, int x)`

Fill one complete frame column using ceiling color above the midline and floor color below it.

Called by: _render_3d_ray
Calls: No project functions

source/render/render_wall.c:15 • file-local helper

`static void _render_column_setup(t_game *g, t_raycast *rc, t_column *col)`

Select texture, clamp distance, project and clip wall height, calculate fractional wall coordinate and horizontal texture flip.

Called by: _render_3d_ray
Calls: texture_for_hit | Library/system: fmax, fmin, floor

source/render/render_wall.c:34 • file-local helper

`static void _render_wall(t_game *g, int x, t_column *col)`

Copy the selected texture column into one clipped screen column. Vertical sampling is based on the original unclipped top.

Called by: _render_3d_ray
Calls: No project functions

source/render/render_wall.c:55 • file-local helper

`static void _render_3d_ray(t_game *g, int i)`

Generate/cast one ray, prepare a column if it hits, then paint background and optional wall across that ray's screen strip.

Called by: render_3d
Calls: raycast_camera_ray, raycast_cast, _render_column_setup, _render_background, _render_wall

source/render/render_wall.c:76 • file-local helper

`void render_3d(t_game *g)`

Process every configured camera sample to fill the 3D image; no window blit occurs here.

Called by: screen_render
Calls: _render_3d_ray

source/render/render_wall.c:96 • public function

> **SPEAKER NOTE** Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

runtime.c

`int runtime_args(int argc, char **argv)`

Require argc=2 and exact lowercase .cub suffix; print usage and return 1 on invalid arguments, otherwise 0.

Called by: main
Calls: common_error | Library/system: ft_strlen, ft_strcmp

source/runtime.c:15 • public function

`void runtime_cleanup(t_game *g)`

Destroy texture/frame images, window, display/context, then parser-owned map allocations; reset graphics handles.

Called by: runtime_setup, runtime_run
Calls: texture_cleanup, parser_cleanup | Library/system: mlx_destroy_image, mlx_destroy_window, mlx_destroy_display, free

source/runtime.c:29 • public function

`int runtime_setup(t_game *g, char *map_path)`

Zero the entire game; parse map and create graphics with short-circuit failure cleanup; set initial needs_redraw. Return 0 success / 1 failure.

Called by: main
Calls: parser_load_map, setup_game, runtime_cleanup

source/runtime.c:47 • public function

`int runtime_run(t_game *g)`

Enter mlx_loop, clean up after it returns, and return the stored application exit status.

Called by: main
Calls: runtime_cleanup | Library/system: mlx_loop

source/runtime.c:59 • public function

`int runtime_close(t_game *g)`

Request loop termination with mlx_loop_end. Return 0 as a callback result; actual cleanup occurs later.

Called by: _game_render, game_frame, hook_press, MLX close event
Calls: No project functions | Library/system: mlx_loop_end

source/runtime.c:66 • public function

> **SPEAKER NOTE** Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

screen.c

`static t_viewport _screen_overlay(t_game *g)`

Fit map aspect ratio within a 256x256 maximum, enforce at least one pixel per viewport dimension and position at top-right margin.

Called by: screen_render
Calls: No project functions | Library/system: fmin

source/screen.c:15 • file-local helper

`void screen_toggle_view(t_game *g)`

Switch V_3D and V_MINIMAP, then request redraw.

Called by: hook_press
Calls: No project functions

source/screen.c:33 • public function

`int screen_clear(t_game *g)`

Check the game/frame address and zero stride*height bytes. Return 0 success / 1 invalid state.

Called by: _game_render
Calls: No project functions | Library/system: ft_memset

source/screen.c:42 • public function

`int screen_render(t_game *g)`

Check frame/window; choose 3D plus overlay or full map, draw map layers, then submit the frame through MLX. Return 0 success / 1 invalid state.

Called by: _game_render
Calls: render_3d, _screen_overlay, render_map_view | Library/system: mlx_put_image_to_window

source/screen.c:51 • public function

SPEAKER NOTE Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

setup.c

`static int _setup_mlx(t_game *g)`

Create the display context and 1024x768 window; report and return 1 if either creation fails, otherwise 0.

Called by: setup_game
Calls: common_error | Library/system: mlx_init, mlx_new_window

source/setup.c:15 • file-local helper

`static void _setup_hook(t_game *g)`

Register five window event callbacks and the per-loop game_frame callback, passing g to each.

Called by: setup_game
Calls: No project functions | Library/system: mlx_hook, mlx_loop_hook

source/setup.c:27 • file-local helper

`static int _setup_frame(t_game *g)`

Create the frame image, borrow its data/format information and clear it; return 1 with an error if image/address acquisition fails.

Called by: setup_game
Calls: common_error | Library/system: mlx_new_image, mlx_get_data_addr, ft_memset

source/setup.c:40 • file-local helper

`int setup_game(t_game *g)`

Create MLX/window, textures and frame in order; stop on failure. Register callbacks after resources succeed. Return 0 success / 1 failure.

Called by: runtime_setup
Calls: _setup_mlx, texture_load, _setup_frame, _setup_hook

source/setup.c:58 • public function

SPEAKER NOTE Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

texture.c

`static int _texture_load(t_game *g, t_img *img, char *path)`

Decode one XPM into an MLX image and acquire its buffer; require address, positive dimensions and 32 bpp. Return 0 success / 1 failure.

Called by: texture_load
Calls: common_error | Library/system: mlx_xpm_file_to_image, mlx_get_data_addr

source/texture.c:15 • file-local helper

`int texture_load(t_game *g)`

Load all four map texture paths into graphics texture slots, stopping at first error. Return 0 success / 1 failure.

Called by: setup_game
Calls: _texture_load

source/texture.c:28 • public function

`void texture_cleanup(t_game *g)`

Destroy any allocated texture image through MLX and zero every texture slot; safe for partially loaded sets.

Called by: runtime_cleanup
Calls: No project functions | Library/system: mlx_destroy_image

source/texture.c:42 • public function

`t_img *texture_for_hit(t_game *g, t_raycast *rc)`

Select WE/EA on X hits or NO/SO on Y hits according to ray sign; return a borrowed texture image pointer.

Called by: _render_column_setup
Calls: No project functions

source/texture.c:55 • public function

SPEAKER NOTE Reading key: signatures and call edges are extracted from the source snapshot. Callback edges are identified separately. Calls are listed once per function; branch order is explained in the walkthrough.

Library boundaries and parser-only test entry

Boundary	Project-relevant behavior
MiniLibX lifecycle	mlx_init / mlx_new_window create context/window; mlx_loop dispatches X11 events and the loop callback; mlx_loop_end sets a termination flag.
MiniLibX image I/O	mlx_new_image and mlx_xpm_file_to_image create images; mlx_get_data_addr returns borrowed pixels; mlx_put_image_to_window presents the frame.
MiniLibX release	mlx_destroy_image, mlx_destroy_window, mlx_destroy_display; the project then frees the allocated MLX context.
libft used here	ft_strlen, ft_strcmp/ft_strncmp, ft_strtrim; ft_memcpy, ft_memset, ft_calloc; ft_abs; ft_putendl_fd. String trimming allocates a new string.
System and math	open/read/close for input; malloc/free for ownership; gettimeofday for time; floor/fabs/fmin/fmax/sqrt/sin/cos for geometry.
tests/map_check.c:main	Accept map path or --textures path; zero game; parser_load_map; optionally access(R_OK) for four paths; parser_cleanup; return 0 accepted / 1 rejected.

The reference covers all authored application functions under source/. Vendored libft and MLX internals are dependencies; their unrelated functions are outside the presentation scope.

SPEAKER NOTE The test harness intentionally avoids graphics initialization. A readable texture file can still fail decoding in a real run.

Final rehearsal: explain the project without the slides

- In 30 seconds: explain how a 2D grid becomes a first-person view.
- In 2 minutes: trace main through parser ownership, setup, callbacks, and cleanup.
- In 3 minutes: work the sample ray from (3.5,2.5) through DDA, wall-face selection, texture sampling, and screen strip.
- In 2 minutes: explain dirty rendering, time scaling, diagonal normalization, and four-corner collision.
- In 2 minutes: explain a malformed file, a graphics initialization failure, and a normal window close.
- From any reference page: identify a function’s caller, input state, result, ownership effects, and next stage.

Completion standard: you can connect every project function to startup, parsing, input/update, rendering, or teardown, and explain the important branches without inventing behavior.

SPEAKER NOTE Keep the code available during rehearsal. Verify names and assumptions when you are unsure; this document is a snapshot, not an automatic guarantee after future edits.